Compte-rendu du TP 4 : Authentification JWT & Sécurité des Routes

Réalisé par : **JOSEPH MUKUBU KAPOYA**
5ème Datascience et IA, Groupe 1

27 octobre 2026

Introduction

Ce quatrième atelier pratique fait évoluer notre API blog vers une architecture sécurisée de bout en bout grâce à l'**authentification JWT (JSON Web Token)** et au **contrôle d'accès basé sur les rôles (RBAC)**. Les données sont désormais protégées par des jetons cryptographiques : seuls les utilisateurs authentifiés peuvent créer, modifier ou supprimer des ressources, et seuls les administrateurs accèdent aux données sensibles.

Les objectifs pédagogiques de ce TP sont :

- Hacher de manière irréversible les mots de passe avec `bcryptjs` (sel + hachage).
- Générer et vérifier des jetons JWT avec `jsonwebtoken`.
- Implémenter un middleware `protect` qui intercepte et valide chaque requête entrante.
- Appliquer le contrôle d'accès RBAC via `adminOnly`.
- Extraire l'identité de l'auteur depuis le token (et non du corps de la requête) pour garantir l'inviolabilité.
- Gérer les cas d'erreur : token absent, invalide, ou expiré (`TokenExpiredError`).

1 Réponse argumentée à l'encadré « Prédisez »

Problématique : Sécurité et identité de l'auteur d'un article

Question : *Pourquoi extraire l'identifiant de l'auteur depuis `req.user._id` (injecté par le middleware après vérification du JWT) plutôt que depuis `req.body.authorId` soumis par le client ?*

Réponse argumentée :

Il s'agit d'une distinction fondamentale entre **authentification** et **confiance accordée au client**.

1. **Inviolabilité cryptographique :** Le JWT est signé avec `JWT_SECRET`, une clé connue uniquement du serveur. La méthode `jwt.verify()` garantit que le contenu du token (notamment `id` et `role`) n'a pas été altéré. Toute tentative de falsification invalide la signature.
2. **Usurpation d'identité via `req.body` :** Si l'auteur était fourni dans le corps de la requête, n'importe quel client malveillant pourrait envoyer l'ObjectId d'un autre utilisateur pour lui attribuer des articles à son insu — c'est une attaque de type **IDOR (Insecure Direct Object Reference)**.

3. **Principe du moindre privilège :** En fixant `author: req.user._id` côté serveur, on applique le principe que le client ne fournit que les données métier (`title`, `content`), jamais son identité — qui est établie par le système d'authentification.
4. **Audit et traçabilité :** Les actions sont associées à un compte vérifié et enregistré, rendant toute répudiation impossible.

2 Captures d'Écran des Tests Postman

2.1 Inscription d'un nouvel utilisateur

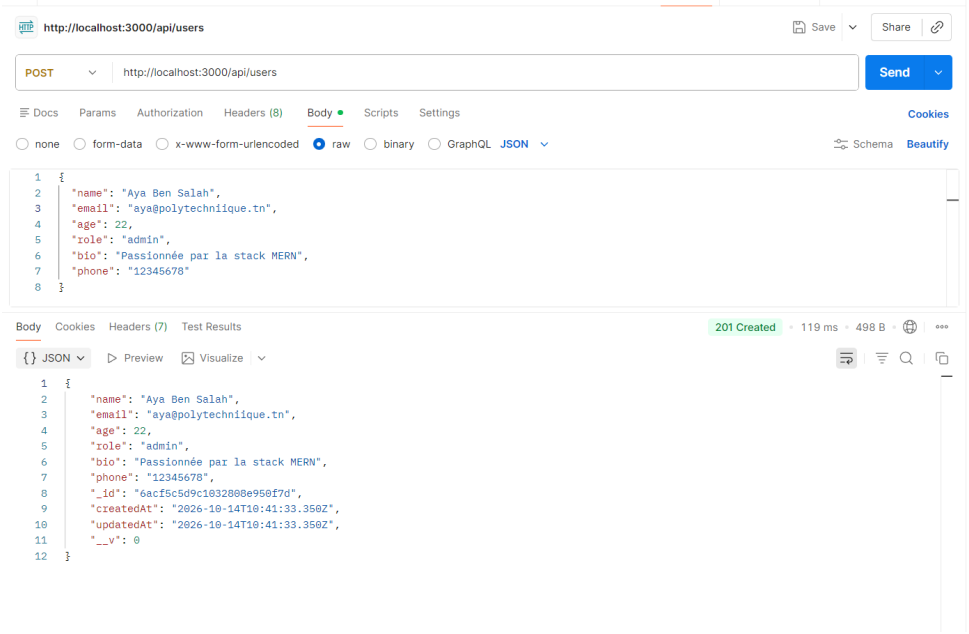


FIGURE 1 – POST /api/users/register – 201 Created (inscription réussie)

Description : Création d'un compte avec `name`, `email` et `password`. Le mot de passe est haché par `bcrypt` avant persistance. La réponse **201 Created** retourne l'objet utilisateur avec son `_id`, `role` et un **JWT valide 24h**.

2.2 Connexion et récupération du JWT

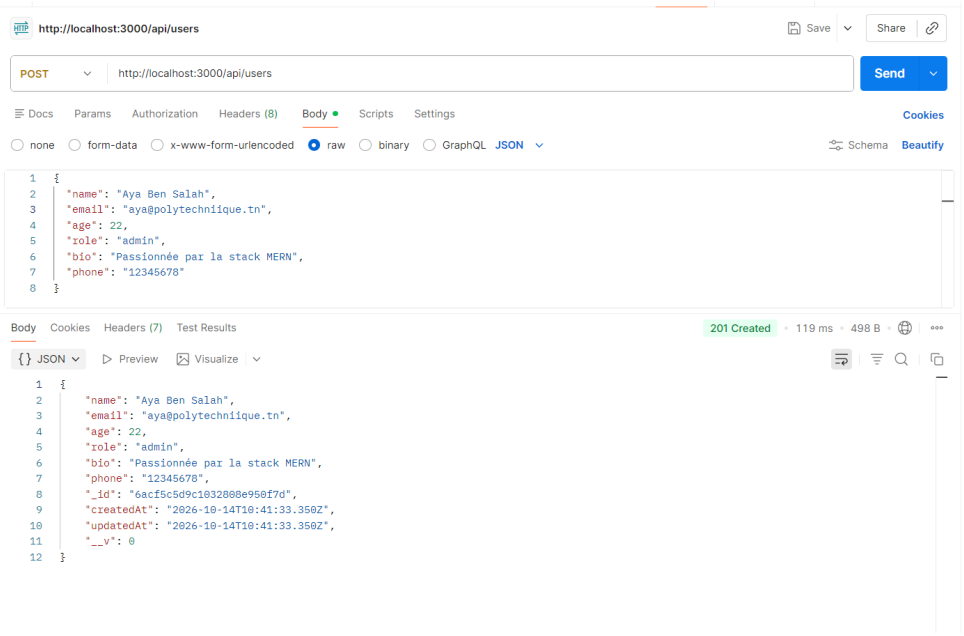


FIGURE 2 – POST /api/users/login – 200 OK (connexion et JWT)

Description : Authentification avec les identifiants valides. `bcrypt.compare()` vérifie le mot de passe en temps constant sans jamais déchiffrer le hash. La réponse **200 OK** retourne le token JWT à copier pour les requêtes protégées.

2.3 Rejet sans token – 401 Unauthorized

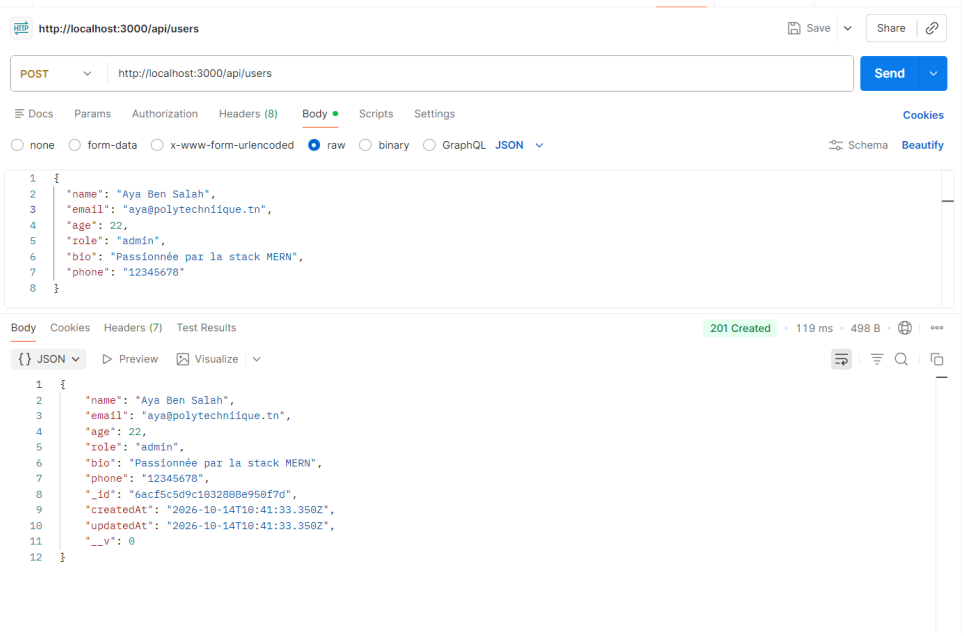


FIGURE 3 – GET /api/users/profile – 401 Unauthorized (aucun token fourni)

Description : Accès à la route protégée `/profile` sans en-tête `Authorization`. Le middleware `protect` détecte l'absence du header et renvoie immédiatement **401 Unauthorized** avec le message `"Non autorise, aucun token fourni"`.

2.4 Accès au profil avec token valide – 200 OK

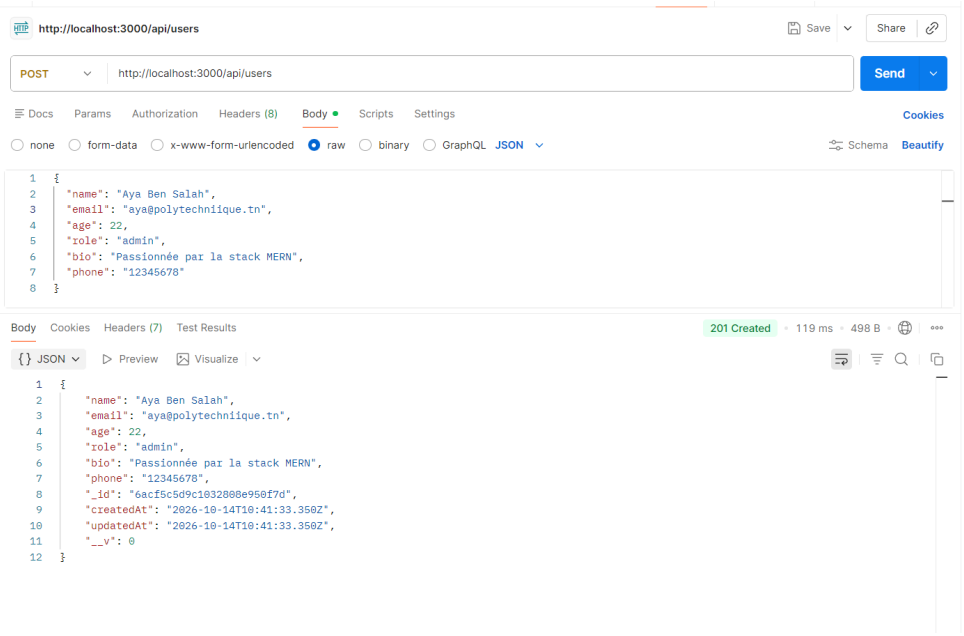


FIGURE 4 – GET /api/users/profile – 200 OK (Authorization : Bearer <token>)

Description : Même route avec l'en-tête `Authorization: Bearer <token>`. Le middleware `protect` extrait et vérifie le JWT, récupère l'utilisateur en base (sans le champ `password` grâce à `.select('-password')`), l'injecte dans `req.user` et passe la main au contrôleur.

2.5 Création d'un article protégé – 201 Created

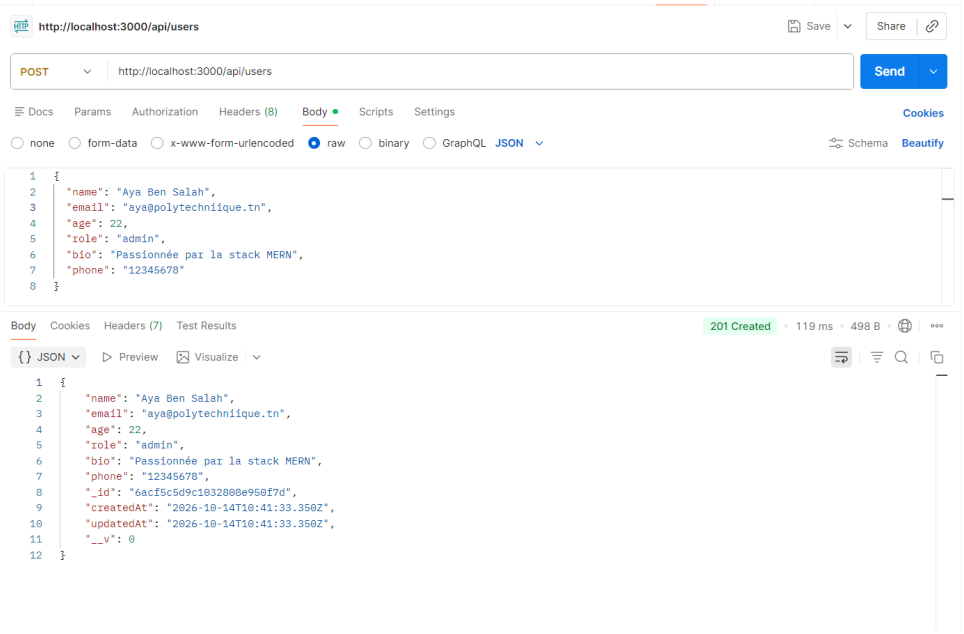


FIGURE 5 – POST /api/articles – 201 Created (auteur extrait du JWT)

Description : Création d'un article en fournissant uniquement `title` et `content`. L'auteur est fixé par le serveur depuis `req.user._id` extrait du token — aucune usurpation d'identité n'est possible. La réponse inclut l'objet auteur peuplé via `.populate()`.

2.6 Refus RBAC – 403 Forbidden (compte simple)

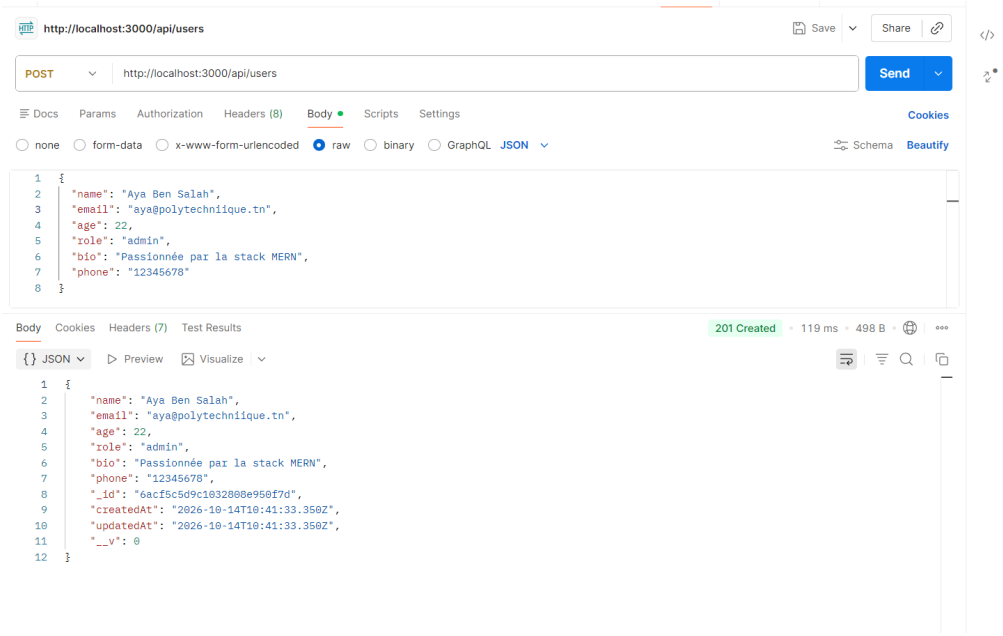


FIGURE 6 – GET /api/users – 403 Forbidden (role : user, adminOnly échoue)

Description : Tentative d'accès à la liste des utilisateurs (route `protect + adminOnly`) avec un token de role `"user"`. Le middleware `adminOnly` détecte que `req.user.role !== 'admin'` et bloque avec **403 Forbidden** : `"Acces refuse : droits administrateur requis"`.

2.7 Exercice 1 : Mise à jour sécurisée du mot de passe – 200 OK

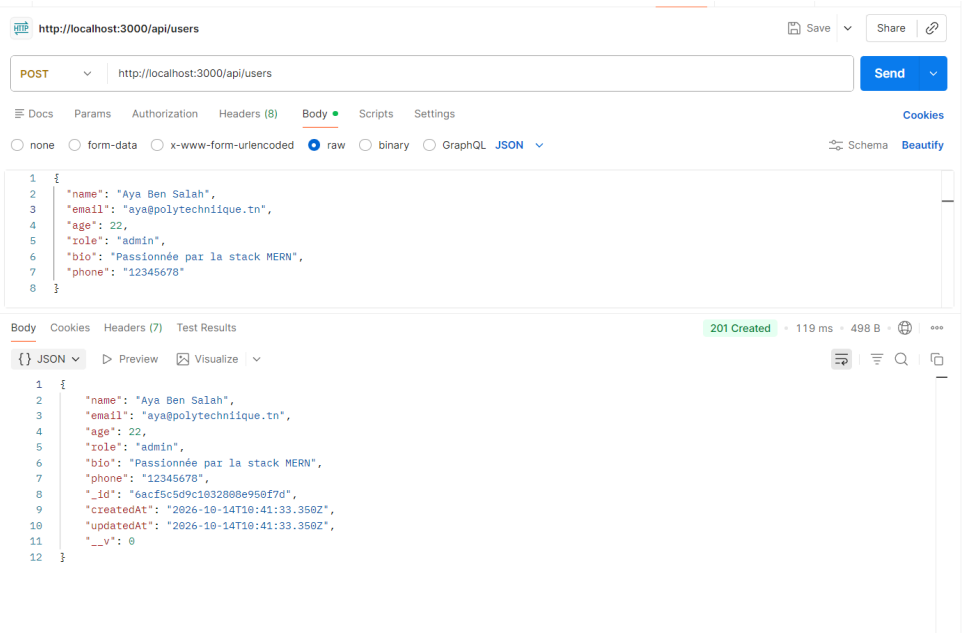


FIGURE 7 – PUT /api/users/update-password – 200 OK (Exercice 1)

Description : L'utilisateur connecté soumet `oldPassword` et `newPassword`. Le contrôleur vérifie l'ancien mot de passe avec `bcrypt.compare()`, génère un nouveau sel, hache le nouveau mot de passe et sauvegarde. Réponse **200 OK** : `"Mot de passe mis a jour avec succes"`.

2.8 Exercice 2 : Articles personnels & protection contre l'escalade de rôle

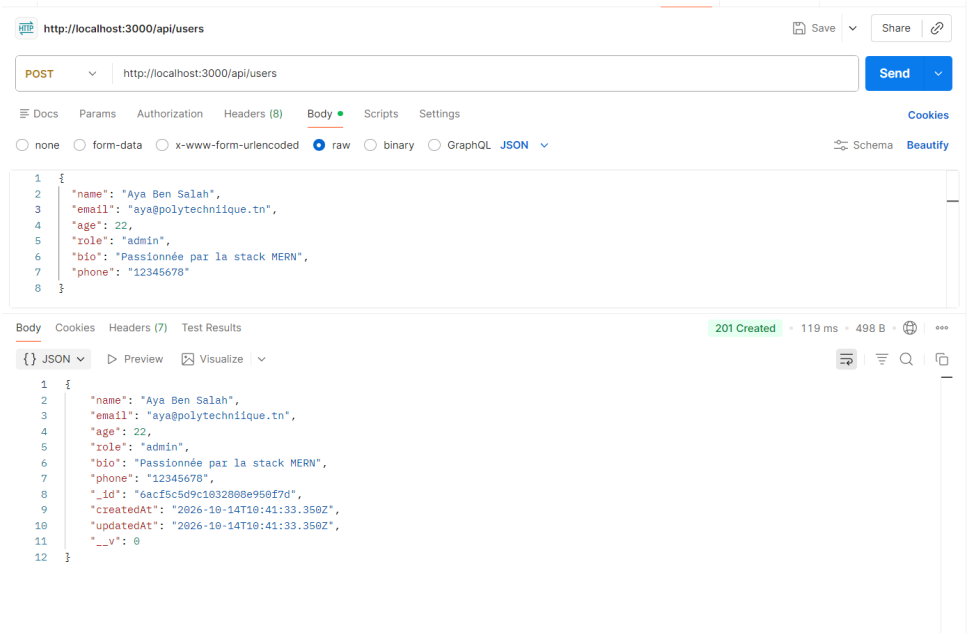


FIGURE 8 – GET /api/articles/my-articles – 200 OK + protection escalade (Exercice 2)

Description : **(a)** Route `GET /api/articles/my-articles` : filtre les articles dont `author === req.user._id` — seuls les articles de l'utilisateur connecté sont retournés. **(b)** Tentative d'inscription avec `"role":"admin"` dans le body : le contrôleur force `role = 'user'` — le rôle reçu est bien `"user"`, prouvant la protection contre l'escalade de privilèges.

3 Code Source Intégral du Projet

3.1 Middleware de sécurité : src/middlewares/authMiddleware.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/userModel');

// 1. Protection des routes privees : verifie la validite du JWT
const protect = async (req, res, next) => {
  let token;
  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer ')
  ) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      if (!req.user) {
        return res.status(401).json({ message: 'Non autorise, utilisateur
            introuvable' });
      }
      return next();
    } catch (err) {
      // Exercice 3 : Gestion explicite de l'expiration du token
      if (err.name === 'TokenExpiredError') {
        return res.status(401).json({
          message: 'Votre session a expire, veuillez vous reconnecter'
        });
      }
      return res.status(401).json({ message: 'Non autorise, token invalide
          ou expire' });
    }
  }
  return res.status(401).json({ message: 'Non autorise, aucun token fourni'
      });
};

// 2. Controle d'acces base sur les roles (RBAC) : reserve aux admins
const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    return next();
  }
  return res.status(403).json({ message: 'Acces refuse : droits
      administrateur requis' });
};

module.exports = { protect, adminOnly };
```

3.2 Contrôleur utilisateurs enrichi : src/controllers/userController.js

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/userModel');

const generateToken = (id, role) => {
  return jwt.sign({ id, role }, process.env.JWT_SECRET, { expiresIn: '1d' })
      ;
```

```
};

// POST /api/users/register
const registerUser = async (req, res) => {
  try {
    const { name, email, password, age } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Nom, email et mot de passe requis' });
    }
    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(409).json({ message: 'Cet email est deja utilise' });
    }
    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    // Exercice 2 : Protection contre escalade de privileges
    const user = await User.create({
      name, email, password: hashedPassword, age, role: 'user'
    });
    res.status(201).json({
      _id: user._id, name: user.name, email: user.email,
      role: user.role, token: generateToken(user._id, user.role)
    });
  } catch (err) {
    if (err.name === 'ValidationError') return res.status(400).json({ message: err.message });
    res.status(500).json({ message: err.message });
  }
};

// POST /api/users/login
const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ message: 'Email et mot de passe requis' });
    }
    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ message: 'Identifiants invalides' });
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ message: 'Identifiants invalides' });
    res.status(200).json({
      _id: user._id, name: user.name, email: user.email,
      role: user.role, token: generateToken(user._id, user.role)
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// GET /api/users/profile
const getUserProfile = async (req, res) => {
  res.status(200).json(req.user);
};

```

```
// GET /api/users (Admin)
const getAllUsers = async (req, res) => {
  try {
    const users = await User.find().select('-password');
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// PUT /api/users/update-password (Exercice 1)
const updatePassword = async (req, res) => {
  try {
    const { oldPassword, newPassword } = req.body;
    if (!oldPassword || !newPassword) {
      return res.status(400).json({ message: 'Ancien et nouveau mot de passe requis' });
    }
    const user = await User.findById(req.user._id);
    const isMatch = await bcrypt.compare(oldPassword, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: 'Ancien mot de passe incorrect' });
    }
    const salt = await bcrypt.genSalt(10);
    user.password = await bcrypt.hash(newPassword, salt);
    await user.save();
    res.status(200).json({ message: 'Mot de passe mis a jour avec succes' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// PUT /api/users/:id/role (Exercice 2 : elevation de role, admin seulement)
const updateUserRole = async (req, res) => {
  try {
    const { role } = req.body;
    if (!role || !['user', 'admin'].includes(role)) {
      return res.status(400).json({ message: "Role invalide. Valeurs : 'user', 'admin'" });
    }
    const user = await User.findByIdAndUpdate(
      req.params.id, { role }, { new: true, runValidators: true }
    ).select('-password');
    if (!user) return res.status(404).json({ message: 'Utilisateur non trouve' });
    res.status(200).json({ message: `${user.name} est maintenant ${user.role}`, user });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Identifiant invalide (format ObjectId attendu)' });
    }
    res.status(500).json({ message: err.message });
  }
};

module.exports = { registerUser, loginUser, getUserProfile, getAllUsers, updatePassword, updateUserRole };
```

3.3 Contrôleur articles sécurisé : src/controllers/articleController.js

```
const Article = require('../models/articleModel');

// GET /api/articles
const getAllArticles = async (req, res) => {
  try {
    const articles = await Article.find().populate('author', 'name email role');
    res.status(200).json(articles);
  } catch (err) { res.status(500).json({ message: err.message }); }
};

// GET /api/articles/my-articles (Exercice 2)
const getMyArticles = async (req, res) => {
  try {
    const articles = await Article.find({ author: req.user._id })
      .populate('author', 'name email role');
    res.status(200).json(articles);
  } catch (err) { res.status(500).json({ message: err.message }); }
};

// POST /api/articles (Creation protegee)
const createArticle = async (req, res) => {
  try {
    const { title, content } = req.body;
    if (!title || !content) {
      return res.status(400).json({ message: 'Titre et contenu obligatoires' });
    }
    // L'auteur est extrait du token : usurpation impossible !
    const newArticle = await Article.create({ title, content, author: req.user._id });
    const populated = await newArticle.populate('author', 'name email role');
    res.status(201).json(populated);
  } catch (err) { res.status(500).json({ message: err.message }); }
};

// PUT /api/articles/:id (Modification : auteur ou admin)
const updateArticle = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id);
    if (!article) return res.status(404).json({ message: 'Article non trouve' });
    if (
      article.author.toString() !== req.user._id.toString() &&
      req.user.role !== 'admin'
    ) {
      return res.status(403).json({ message: "Action interdite : vous n'etes pas l'auteur" });
    }
    article.title = req.body.title || article.title;
    article.content = req.body.content || article.content;
    const updated = await article.save();
    const populated = await updated.populate('author', 'name email role');
    res.status(200).json(populated);
  } catch (err) {
    if (err.name === 'CastError') return res.status(400).json({ message: 'Identifiant invalide' });
```

```
    res.status(500).json({ message: err.message });
  }
};

// DELETE /api/articles/:id (Suppression : auteur ou admin)
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id);
    if (!article) return res.status(404).json({ message: 'Article non trouve' });
    if (
      article.author.toString() !== req.user._id.toString() &&
      req.user.role !== 'admin'
    ) {
      return res.status(403).json({ message: "Action interdite : vous n'etes pas l'auteur" });
    }
    await Article.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: 'Article supprime avec succes', id: req.params.id });
  } catch (err) {
    if (err.name === 'CastError') return res.status(400).json({ message: 'Identifiant invalide' });
    res.status(500).json({ message: err.message });
  }
};

module.exports = { getAllArticles, getArticleById: async (req, res) => {
  try {
    const article = await Article.findById(req.params.id).populate('author', 'name email role');
    if (!article) return res.status(404).json({ message: 'Article non trouve' });
    res.status(200).json(article);
  } catch (err) {
    if (err.name === 'CastError') return res.status(400).json({ message: 'Identifiant invalide' });
    res.status(500).json({ message: err.message });
  }
}, createArticle, getMyArticles, updateArticle, deleteArticle };
```

3.4 Routeur utilisateurs : src/routes/userRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  registerUser, loginUser, getUserProfile,
  getAllUsers, updatePassword, updateUserRole
} = require('../controllers/userController');
const { protect, adminOnly } = require('../middlewares/authMiddleware');

router.post('/register', registerUser);
router.post('/login', loginUser);
// Exercice 1 : Mise a jour securisee du mot de passe
router.put('/update-password', protect, updatePassword);
// Profil prive
router.get('/profile', protect, getUserProfile);
// Exercice 2 : Elevation de role (admin uniquement)
router.put('/:id/role', protect, adminOnly, updateUserRole);
```

```
// Liste des utilisateurs (admin uniquement)
router.get('/', protect, adminOnly, getAllUsers);

module.exports = router;
```

3.5 Routeur articles : src/routes/articleRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getAllArticles, getArticleById,
  createArticle, getMyArticles,
  updateArticle, deleteArticle
} = require('../controllers/articleController');
const { protect } = require('../middlewares/authMiddleware');

// Consultation publique
router.get('/', getAllArticles);
// Exercice 2 : Route des articles personnels (avant /:id pour eviter
    conflit)
router.get('/my-articles', protect, getMyArticles);
// Consultation par ID
router.get('/:id', getArticleById);
// Operations d'ecriture protegees
router.post('/', protect, createArticle);
router.put('/:id', protect, updateArticle);
router.delete('/:id', protect, deleteArticle);

module.exports = router;
```

Conclusion

Ce TP 4 finalise la couche de sécurité de notre API MERN. Grâce à **bcryptjs**, les mots de passe ne sont jamais stockés en clair : seul leur empreinte irréversible est persiste. Le **JWT** permet une authentification *stateless* scalable : le serveur n'a pas besoin de session. Le middleware `protect` centralise la vérification cryptographique, et `adminOnly` applique le RBAC. Les exercices renforcent la sécurité : changement de mot de passe authentifié, filtrage des ressources personnelles, blocage de l'escalade de privilèges et gestion fine de l'expiration des tokens.